

1.5 Aritmética: operadores de igualdade e operadores relacionais

A maioria dos programas realiza cálculos aritméticos. Os operadores aritméticos são resumidos na Figura 1.5.

Operação Java	Operador	Expressão algébrica	Expressão Java
Adição	+	$f + 7$	$f + 7$
Subtração	-	$p - c$	$p - c$
Multiplicação	*	bm	$b * m$
Divisão	/	x/y ou $\frac{x}{y}$ ou $x \div y$	x / y
Resto	%	$r \bmod s$	$r \% s$

Figura 1.5: Tabela de operadores aritméticos.

Fonte: Deitel and Deitel (2015)

O Java possui Regras de precedência para operadores semelhante a C. A Figura 1.6 determina a ordem de precedência dos operadores em Java.

Operador(es)	Operação(ões)	Ordem de avaliação (precedência)
* / %	Multiplicação Divisão Resto	Avaliado primeiro. Se houver vários operadores desse tipo, eles são avaliados da <i>esquerda para a direita</i> .
+ - =	Adição Subtração Atribuição	Avaliado em seguida. Se houver vários operadores desse tipo, eles são avaliados da <i>esquerda para a direita</i> . Avaliado por último.

Figura 1.6: Tabela de precedência de operadores.

Fonte: Deitel and Deitel (2015)

Os operadores relacionais e de igualdade são utilizadas para comparar variáveis e condições. A Figura 1.7 mostra os operadores disponíveis em Java.

Operador algébrico	Operador de igualdade ou relacional Java	Exemplo de condição em Java	Significado da condição em Java
<i>Operadores de igualdade</i>			
=	==	$x == y$	x é igual a y
≠	!=	$x != y$	x é não igual a y
<i>Operadores relacionais</i>			
>	>	$x > y$	x é maior que y
<	<	$x < y$	x é menor que y
≥	>=	$x >= y$	x é maior que ou igual a y
≤	<=	$x <= y$	x é menor que ou igual a y

Figura 1.7: Tabela de operadores.

Fonte: Deitel and Deitel (2015)

Operadores de atribuição compostos abreviam expressões de atribuição. Por exemplo, o operador `+=` adiciona o valor da expressão na sua direita ao valor da variável na sua esquerda e armazena o resultado na variável no lado esquerdo do operador. A Figura 1.8 resume as possíveis atribuições compostas.

Operador de atribuição	Expressão de exemplo	Explicação	Atribuições
<i>Suponha:</i> <code>int c = 3, d = 5, e = 4, f = 6, g = 12;</code>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 a c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 a d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 a e
<code>/=</code>	<code>f /= 3</code>	<code>f = f / 3</code>	2 a f
<code>%=</code>	<code>g %= 9</code>	<code>g = g % 9</code>	3 a g

Figura 1.8: Tabela de operadores compostos.

Fonte: Deitel and Deitel (2015)

Java oferece dois operadores unários de incremento e decremento, `++` e `--` respectivamente. Eles podem ser utilizados com pré e pós dependendo se vem antes ou depois da variável. A Figura 1.9 resume as possibilidades de utilização dos operadores unários.

Operador	Nome do operador	Exemplo	Explicação
<code>++</code>	pré-incremento	<code>++a</code>	Incrementa a por 1, então utiliza o novo valor de a na expressão em que a reside.
<code>++</code>	pós-incremento	<code>a++</code>	Usa o valor atual de a na expressão em que a reside, então incrementa a por 1.
<code>--</code>	pré-decremento	<code>--b</code>	Decrementa b por 1, então utiliza o novo valor de b na expressão em que b reside.
<code>--</code>	pós-decremento	<code>b--</code>	Usa o valor atual de b na expressão em que b reside, então decrementa b por 1.

Figura 1.9: Tabela de operadores unários.

Fonte: Deitel and Deitel (2015)

1.6 Tomada de decisão

Uma condição é uma expressão que pode ser **true** ou **false**. O exemplo de instrução de seleção `if` do Java que permite a um programa tomar uma decisão com base no valor de uma condição. Se a condição em uma estrutura `if` for verdadeira, o corpo da estrutura `if` é executado. Se a condição for falsa, o corpo não é executado (Deitel and Deitel (2015)).

Pode-se ainda, acrescentar a execução de comandos quando a estrutura `if` for falsa com o comando **else**. Por fim, é possível aninhar comando `if` e **else**, fazendo uma cadeia de comando condicionais.

Modifique o projeto anterior para executar o exemplo abaixo.

```
import java.util.Scanner;
public class EntradaSaida {
    public static void main(String[] args) {
```

```
Scanner teclado = new Scanner(System.in);
int number1; //declarando as variaveis
int number2;

System.out.println("Digite o primeiro numero");
number1 = teclado.nextInt();
System.out.println("Digite o segundo numero");
number2 = teclado.nextInt();

//Comando if e comparacao de igualdade
if(number1 == number2){
    System.out.println("Os numeros sao iguais");
}else{
    System.out.println("Os numeros sao diferentes");
}
if(number1 < number2){
    System.out.println("Os numero 1 e maior que o
        numero 2");
} else{
    System.out.println("Os numero 2 e maior que o
        numero 1");
}
//Esse if sera executado independente do resultados dos
    anteriores
if (number1 >= number2)
    System.out.printf("numero 1 e maior igual ao numero 2
        %d >= %d", number1, number2);
teclado.close();
}
}
```

1.6.1 O comando switch

O **switch** em Java é uma estrutura de controle de fluxo utilizada para selecionar um bloco de código entre vários possíveis, com base no valor de uma variável. Ele é frequentemente usado como uma alternativa mais limpa ao **if-else-if** encadeado, especialmente quando você está comparando uma variável contra múltiplos valores constantes.

Veja o exemplo de switch para Java 14+ e sua sintaxe:

```
int dia = 3;
String nomeDoDia = switch (dia) {
    case 1 -> "Segunda-feira";
    case 2 -> "Terca-feira";
    case 3 -> "Quarta-feira";
    default -> "Dia invalido";
};
System.out.println(nomeDoDia);
```

1.7 Estruturas de repetição

Em Java, os laços de repetição (loops) são fundamentais para automatizar tarefas repetitivas. Tanto o `for` quanto o `while` servem para repetir um bloco de código, mas a escolha entre eles depende de como você controla a repetição.

O `for` é geralmente utilizado quando você sabe exatamente quantas vezes o código deve repetir. Ele condensa a inicialização, a condição e a atualização em uma única linha. Veja a sintaxe abaixo, muito semelhante a C.

```
for(inicializacao; condicao; atualizacao){
    //comandos;
}
while(condicao){
    //comandos;
    //comando para finalizacao;
}
```

1.7.1 A estrutura for-each

O for-each (oficialmente chamado em Java de *Enhanced For Loop*) é uma das ferramentas mais utilizadas na linguagem. Ele foi introduzido no Java 5 com um objetivo principal: simplificar a leitura de coleções e **arrays**. Em vez de se preocupar com índices, o `for-each` foca no elemento em si. Funciona para cada elemento dentro da coleção.

Veja o exemplo abaixo, a saída serão os nomes em ordem da lista:

```
...
String[] alunos = {"Ana", "Bia", "Caio"};
for (String nome : alunos) {
    System.out.println(nome);
}
```

A Tabela 1.1 pode ajudar na decisão de qual estrutura utilizar entre `for` e `for-each`

Cenário	Melhor escolha	Motivo
Percorrer toda a lista	for-each	É mais legível e menos propenso a erros de índice.
Precisar do índice (i)	for comum	O for-each não te dá acesso à posição (ex: i=0).
Modificar a estrutura	for comum	Tentar remover itens de uma lista usando for-each causa erro.
Percorrer ordem inversa	for comum	O for-each sempre percorre do primeiro ao último.

Tabela 1.1: Comparativo: `for` tradicional vs. `for-each`.

1.8 Arrays

Um array é um grupo de variáveis (chamados elementos ou componentes) que contém valores todos do mesmo tipo. Os arrays são objetos; portanto, são considerados tipos por referência Deitel and Deitel (2015).

1.8.1 Declarando arrays

Assim como outros objetos, os arrays ocupam espaço na memória e são instanciados por meio da palavra-chave `new`. Para criar um array, é necessário especificar o tipo dos seus elementos e a quantidade desejada, definindo essas informações diretamente na expressão de criação (Deitel and Deitel (2015)). Isso tudo porque podemos trabalhar com arrays de qualquer tipo de dado, desde tipos primitivos até objetos complexos, e é nos objetos complexos que a declaração de Arrays dessa forma se mostram úteis.

```
int[] c; // declara a variavel de array  
c = new int[12]; // cria o array; atribui a variavel de array
```

2 Referências Bibliográficas

Deitel, H. M. and Deitel, P. J. (2015). *Cómo programar en Java*. Pearson educación.